

Javascript

▼ Fundamentals

▼ Introduction

- Initially, websites were static (only HTML and CSS)
- Javascript adds behavior to web pages.

▼ Defination

Javascript is hight - level, interpreted programming language used to make web pages interactive and dynamic.

Example

```
<button onclick="alert('hello student') ">Click Me</button>
<!-- when the button is clicked, js runs and shows a popup
-->
```

Note:

1. **High-level** : Easy for humans to read and write
2. **Interpreted**: Runs line by line in the browser and No need to compile before running
3. **Interactive**: User can interact with the page (Clicking a button, Filling a form, playing a video etc)
4. **Dynamic** : Content changes without reloading the page (live search results, updating cart count etc)

Where JavaScript is used ?

1. web development(Frontend and Backend)
2. Mobile Apps
3. Game Development etc..

▼ History

- JavaScript was created by Brendan Eich in 1995

- Developed in just 10 days
- Originally named : Mocha → LiveScript → Javascript
- Ryan Dahl created Node.js, which allows JavaScript to run outside the browser.

Why "JavaScript" ?

- Name Inspired by Java (For marketing Purposes)
- But JavaScript and Java are completely different

Standardization

Standardized as EcmaScript (ES) and managed by ECMA International (European computer Manufacturing Association)

Note:

- ES6 (2015) → Major update (let, const, arrow functions...)

▼ Key features of JavaScript

1. **Interpreted:** Runs line by line, no compilation needed and runs directly in the browser
2. **Dynamic Typing:** No need to declare variable type

```
let data = "Hello";
data = 100; // allowed
```

3. **Event - Driven programming:** code runs based on user actions

```
<button onclick="alert('welcome to js')">click</button>
```

4. **Synchronous and Asynchronous Behaviour:**

- a. **Synchronous: code runs line by line**

```
console.log("1");
console.log("2");
console.log("3");
```

- b. **Asynchronous:** Some tasks run later without stopping main execution

```
console.log("1");  
setTimeout(()=> {console.log("2")}, 2000)  
console.log("3");
```

5. **Platform Independent:** runs everywhere in all operating systems and inside browser and on server (Node.js)

Note:

Scripting Language vs programming language

- A scripting Language is used to automate tasks and usually run inside another environment (like a browser) where as A programming language is used to build complete applications/software
- JavaScript is primarily a Scripting Language but it is also considered a full-fledged programming language (because of Node.js)

▼ Ways to Adding JS

Javascript can be added to HTML in three ways:

▼ Inline JavaScript

JavaScript is written inside HTML tags

```
<button onclick="alert('Hello') "></button>
```

▼ Internal JavaScript

JavaScript is written inside `<script>` tag within HTML

```
<!DOCTYPE html>  
<html>  
<head></head>  
  <body>  
    <script>  
      console.log("hello internal js")  
    </script>  
  </body>  
</html>
```

▼ External Javascript

JavaScript is written in a separate .js file

```
<!DOCTYPE html>
<html>
<head></head>
  <body>
    <script src="./main.js"></script>
  </body>
</html>
```

```
// main.js
console.log("Hello world js");
```

▼ Comments

- Comments are lines in the code that are not executed by js
- They are used to explain code and improve readability

▼ Types of Comments in js

1. **Single line comments** : used to write a comment in one line

Example : This is a single line comment

```
// this is single line comment
```

2. **Multi - Line comment** : Used to write a comments in multiple line

```
/* this is
multi line
comment */
```

▼ JS Engine

- A JavaScript engine is a program that executes javascript code
- It converts your javascript code into something the computer can understand and run it

→ Real- time example of js examples

1. v6 - used in chrome

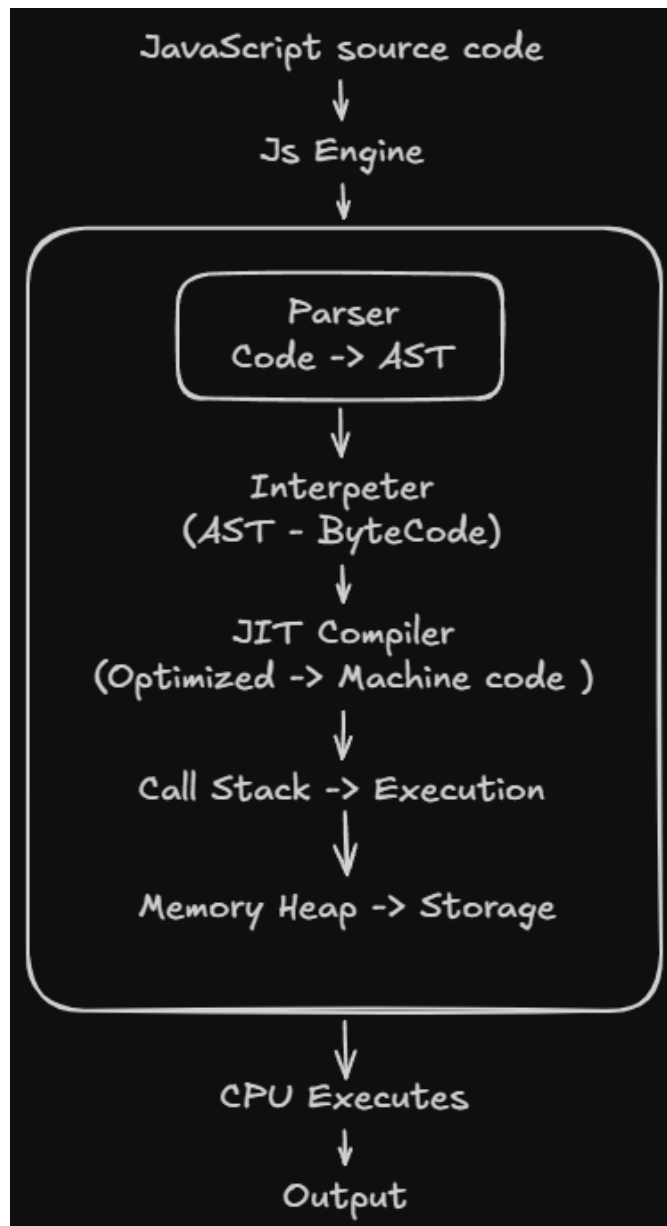
2. SpiderMonkey - used in fireFox

3. JavaScriptCore - used in safari

▼ Why Js engine is needed

Browsers don't understand javascript directly, so Js engine acts as a translator + executor

▼ How Js engine works



1. You write code in a file or browser

2. Browser send code to js engine, Where engine starts processing

3. Parser → code is read line by line, syntax is checked and converted into AST(Abstract Syntax Tree)
4. Interpreter → AsT converted into ByteCode and execution starts immediately
5. JIT compiler → optimized inot machine code
6. Memory allocation → variables & objects stored in Heap While Function are pushed into call stack and exectues in order (LIFO)
7. Machine code is sent to CPU and perform operators
8. Result shown in Browser console or webpage UI

Java Script code -> parser -> converted to AST -> Interpreted to Bytecode -> optimized by JIT -> stored in memory -> executed using call stack -> output shown

▼ Variables

A variable is a combines used to store data

```
let name = "hello";  
// name -> variable name  
// "hello" -> value inside the variable
```

Declaration V/S Initialization in JavaScript

Declaration means creating a variable (reserving memory)

```
let name;
```

Initialization it means assigning a value to a variable

```
name = "hello";
```

Declaration + Initialization

```
let name = "hello";
```

▼ How to Declare variables

In JavaScript, we use 3 keywords

1. var (old way)

```
var age = 25;
```

2. let (modern & recommended)

```
let age = 25;
```

3. const (constant value)

```
const pi = 3.14;
```

▼ Tokens

Tokens are the smallest individual units (piece) of a Javascript program that have meaning.

▼ Types of Tokens in JavaScript

There are mainly 5 types of tokens

▼ 1. keywords

- Keywords are reserved words in Javascript that already have predefined meanings.
- Should always be written in lowercase

example : `let` , `var` , `const` , `if` , `else` , `for` , `while` etc.

▼ 2. Identifiers

Identifiers are the name given to variables, function, classes, arrays, objects

▼ Rules for identifiers

1. cannot starts with number but allowed after first character
2. can't use keywords
3. except `_` and `$`, not other special character are allowed

Example : `name` , `age` , `salary` etc

▼ 3. Literals

Literals are values directly written in the program.

Example : `10` , `"hello"` , `true` etc..

▼ 4. Operators

Operators are predefined symbols used to perform operations

example : +, -, *, /, =, > etc..

▼ 5. Separators

Separators are symbols used to separate code parts

Example: (), {}, [] etc..

▼ DataTypes

A Data type specifies the type of value a variable can store

▼ Primitive Data Type :

▼ Types of Primitive Data Types:

1. Number

2. String

3. Boolean

4. Undefined

When a variable is declared but not assigned a value, JavaScript automatically assigns undefined.

In simple words, it represents uninitialised variable.

```
let age;  
console.log(age); //undefined  
console.log(typeof undefined); //undefined
```

5. Null

Represents intentional absence of value.

```
let data = null;  
console.log(typeof null); //object (This is a historical bug in JavaScript).
```

6. BigInt

Used to store very large integers beyond Number limit($2^{53}-1$).

Note: The n at the end represents BigInt.

```
let accountNo = 12345678912345612345n  
console.log(accountNo) //12345678912345612345n
```

```
console.log(typeof accountNo) //bigint
```

7. Symbol

Used to create unique values.

```
let name = Symbol("Pavan")
let coreJavaTrainer = Symbol("Pavan")
console.log(name) //Symbol(Pavan)
console.log(typeof name) //symbol
console.log(name == coreJavaTrainer) //false
```

▼ **Note:** `typeof` operator

`typeof` operator is used to check the data type of a value

```
typeof 10
typeof "hello"
typeof true
```

▼ Non-Primitive Data Types

- Non-primitive data types can store multiple values or complex data.
- Mutable (can be changed)
- They are also called Reference Types Because they store reference/address.

▼ Types of Non-Primitive Data Types:

1. Object

Objects store data in key-value pairs.

```
let student = {
  name: "prashant",
  age: 25,
}
console.log(typeof student); //object
```

2. Array

Array stores multiples values into []

```
let colors = ['red', 'blue', 'green']
console.log(typeof colors) //objects ( because arrays are internally objects )
```

3. Function

Functions are reusable blocks of code used to perform a specific task.

```
function greet(){
    console.log("hello");
}
greet()
console.log(typeof greet) // function ( in js, functions are treated as objects)
```

▼ Scopes

The area or region where a variable can be accessed or used.

▼ Types of scope in JavaScript

JavaScript has mainly:

▼ 1. Global Scope

- A variable declared outside all functions and blocks belongs to global scope
- It can be accessed from anywhere in the program.

▼ 2. Function Scope

- Variables declared inside a function can only be accessed inside that function.
- `var` follows Function Scope

▼ 3. Block Scope

- Variables declared using `let` or `const` inside `{ }` are accessible only inside that block.
- `let` and `const` follow Block Scope.

NOTE:

▼ Global Scope vs Script Scope

Variables declared using `var` at the top level become part of the global object (window in browsers) whereas variables declared using `let` and `const` at the top level belong to the script scope and are not attached to the global object

```
var a = 10;
let b = 20;
const c = 30;
console.log(window.a); // 10
console.log(window.b); // undefined
console.log(window.c); // undefined
```

▼ Lexical Scope

Lexical Scope means inner functions can access variables from outer functions.

```
function outer(){
  let userName = "Rahul";
  function inner(){
    console.log(userName); // rahul
  }
  inner()
}
outer()
```

Note: child can access parent variables but parent cannot access child variables.

keyword	Global Scope	Function Scope	Block Scope	Script Scope
var	yes	yes	no	no
let	no	no	yes	yes
const	no	no	yes	yes

▼ Operators

- Operators are special symbols used to perform operations on values and variables
- An operand is the value or variable on which an operator performs an operation

```
//nexample : 10+ 20
```

Here : 10 → Operand, 20 → Operand, + → operator

- An expression is a combination of operands and operators that produces a result.

▼ Types of Operators in JavaScript

▼ 1. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations

Ex : +, -, *, /, %, **

Note:

Rules of (+)

- If one operand is string → Concatenation happens.
- Boolean values convert into numbers.
- i.e true → 1, false → 0
- null behaves like 0 whereas undefined converts to NaN
- Result becomes string if concatenation happens.

Rules of (-, *, /, %)

- Always tries to convert values into numbers and Invalid conversion gives NaN.
- Division by 0 gives Infinity.

▼ Examples for **Arithmetic operators**

```
// Addition ( + )
console.log(10 + 20) //30
console.log(10 + "20") // 1020
console.log("10"+20) // 1020
console.log(10 + true) // 11
console.log(10 + false) // 10
console.log(10 + null) // 10
console.log(10 + undefined) // NaN
console.log(10 + 20+ "JS") // 30Js
console.log("JS"+10+20)// JS1020
console.log(typeof (10+ "20")) // string
```

```
// Subtraction ( - )
console.log(20 - 10) //10
console.log(20 - "10") // 10
console.log("50"- "20") // 30
console.log(20 - true) // 19
console.log(20 - false) // 20
console.log(20 - null) // 20
console.log(20 - undefined) // NaN
console.log(20 - "JS") // NaN
console.log(typeof (20 + "10")) // number
```

```
// Multiplication ( * )
console.log(5 * 5) //25
console.log(5 * "5") // 25
console.log("5" * "2") // 10
console.log(5 * true) // 5
console.log(5 * false) // 0
console.log(5 * null) // 0
console.log(5 * undefined) // NaN
console.log(5 * "JS") // NaN
console.log(typeof (5 * "5")) // number
```

```
// Division ( / )
console.log(10/ 2) //5
console.log(10 / "2") // 5
console.log("20" / "2") // 10
console.log(10 / true) // 10
console.log(10 / false) // Infinity
console.log(10 / null) // Infinity
console.log(10 / undefined) // NaN
console.log(10 / "JS") // NaN
console.log(typeof (10 / "2")) // number
```

```
// Modulus ( % )
console.log(10% 2) //0
console.log(10 % 3)// 1
console.log(10 % "3") // 1
console.log(10 % true) // 0
console.log(10 % false) // NaN
```

```
console.log(10 % null) // NaN
console.log(10 % undefined) // NaN
```

```
// Exponent ( ** )
console.log(2**2) // 4
console.log(10 ** 2) // 100
```

▼ 2. Increment & Decrement Operators

- Used to increase or decrease value by 1

Operator	Meaning
++	Increment
—	Decrement

→ **PRE- INCREMENT (++a)** : First increments then returns value

```
let a = 10
console.log(++a) // 11
console.log(a) // 11
```

→ **POST- INCREMENT (a++)** : First returns value then increments.

```
let b = 20
console.log(b++) // 20
console.log(b) // 21
```

→ **PRE- DECREMENT (—a)** : First decrements then returns value

```
let c = 15
console.log(--c) //14
console.log(c) //14
```

→ **PRE- DECREMENT (a—)** : First returns value then decrements

```
let d = 25
console.log(d--) //25
console.log(d) // 24
```

▼ 3. Comparison / Relational operators

- Used to compare values
- example : `>, <, ≥, ≤, ==, ===, ≠, !==`

Operator	Checks Values	Checks Datatype	Type Conversion
<code>!=</code>	yes	no	yes
<code>!==</code>	yes	yes	no

```
let num1 = 10
let num2 = 20
let num3 = "10"
console.log(num1>num2) //false
console.log(num1<num2) //true
console.log(num1 >= num3) //true
console.log(num1 <= num3) //true
console.log(num1 == num3) //true
console.log(num1 === num3) // false
console.log(num1 != num3) // false
console.log(num1 !== num3) //true
console.log(false == 0) // true
console.log(false === 0) // false
console.log(null == undefined) //true
console.log(null === undefined) //false
console.log("" == 0) //true
console.log("" === 0) //false
```

▼ 4. Logical Operators

- Used to combine conditions
- example : `&&, ||`

▼ `&&`

- `&&` Does not always return `true` or `false`
- It returns, the first falsy value or last truthy value

```
let x = 10
let y = 20
let z = '10'
```

```

console.log(x<y && y>x) // true
console.log(x === z && y>x) // false
console.log(true && true && true) //true
console.log(true && false) //false
console.log(10 && 20) //20
console.log(0 && 20) //0
console.log("hi" && "bye") //bye

```



- || Does not always return `true` or `false`
- It returns, the first truthy value or the last falsy value.

```

let x = 10
let y = 20
let z = '10'
console.log(x>y || y>x) // true
console.log(x===2 || y<x) // false
console.log(10 || 20) // 10
console.log((0 || 20) // 20
console.log("" || "javascript") // javascript

```

▼ 5. Assignment Operators

- Used to assign values
- ex : `=` , `+=` , `-=` , `*=` , `/=` , `%=`

```

// +=
let num = 10
num += 5
console.log(num) // 15
num += "10"
console.log(num) // 1510

// -=
let num = 10
num -= 5
console.log(num) // 5
num -= "10"
console.log(num) // -5

```

```
// *=
let num = 5
num *= 2
console.log(num)//10
num *= '2'
console.log(num) //20

// /=
let num6 = 20
num6 /= 2
console.log(num6); // 10
num6 /= '2'
console.log(num6); // 5

// %=
let num7 = 10
num7 %= 3
console.log(num7); //1
num7 %= '2'
console.log(num7); // 1
```

▼ 6. Ternary Operator

- Shortcut for `if-else`

```
// </> syntax
condition ? trueblock : falseblock

// example
let understand_js = true
understand_js ? console.log("good work") : console.
log("bad work")
```

▼ 7. Unary Operato

- unary operators work with single operand
- `+` , `-`

```
// unary plus (+) : convert value into number
console.log(+ "10") // 10
console.log(typeof + "10") // number
```

```
console.log(+ "hello") // NaN

// unary minus (-) : convert value into negative number
console.log(- "10") // -10
console.log(- true) // -1
console.log(- false) // -0
```

▼ **Note:** What is NaN?

NaN = Not a Number

Occurs when js fails to convert value into valid number

```
// example : console.log(10 - "abc") // NaN
```

▼ What is differences b/w (==) and (===)?

== Checks only values whereas === checks value + datatype

```
console.log(10=="10") // true
console.log(10 === "10") //false
```

▼ **Falsy** Value in JavaScript

Falsy values are the values that becomes false when converted into Boolean

```
1. false
2. 0
3. -0
4. 0n
5. ""
6. null
7. undefined
8. NaN
```

▼ **Decision Statements**

- Decision statements are used to make decisions and control the flow of execution based on conditions.

- They execute different blocks of code depending on whether the condition is `true` or `false`

▼ Types of Decision statements in javascript

▼ `If` Statement

The if statement is used to execute a block of code only when the condition is true

```
// </> syntax

if (condition){
    code
}

// example
let age = 18
if (age>18){
    console.log("Eligible to vote");
}
```

▼ `if else` statement

The if else statement is used to execute one block when the condition is `true` and another block when condition is `false`

```
// </> syntax
if ( condition){
    true block
}
else {
    false block
}

// example
let num = 10
if (num%2==0){
    console.log("even number")
}
else{
```

```
    console.log("odd number")
}
```

▼ else if statement

The else if ladder is used when multiple conditions need to be checked.

```
// </> syntax
if (condition){
    code
}
else if(condition2){
    code
}
else{
    code
}

//Example:
let marks = 75
if(marks >= 90){
    console.log("Grade A")
}
else if(marks >= 75){
    console.log("Grade B")
}
else if(marks >= 50){
    console.log("Grade C")
}
else{
    console.log("Fail")
}
```

▼ nested if statement

Nested if means writing one if statement inside another if statement.

```
// </>Syntax
if(condition1){
    if(condition2){
        code
    }
}
```

```

}

// Example:
let age = 20
let hasLicense = true
if(age >= 18){
    if(hasLicense){
        console.log("Can drive")
    }
}

```

▼ `switch` statement

- The switch statement is used to execute one block of code from multiple cases
- Break is used to stop the execution after matching a case.
- default block executes when no case matches.

```

// syntax
switch(expression){
    case value1:
        code
        break
    case value2:
        code
        break
    default:
        code
}

// example
let day = 2
switch(day){
    case 1:
        console.log("Monday")
        break;
    case 2:
        console.log("tuesday")
        break;
    case 3:
        console.log("wednesday")

```

```

        break;
    default:
        console.log("Invalid Day")
    }

// Example : 2
let amount = 250
switch (true){
    case (amount >= 350):
        console.log("Eat pista house biryani")
        break;
    case (amount >= 350 && amount < 350):
        console.log("eat mehfile biryani")
        break;
    default:
        console.log("drink and sleep")
}

```

▼ Loops

- Loops are used to execute a block of code repeatedly until a condition becomes false.
- They help reduce code repetition and make programs shorter and easier to manage.

▼ Types are mainly 5 types of loops

▼ **for** loop

- the for loop is used when the number of iterations is known.
- It contains 3 parts
 - i. Initialization
 - ii. conditions
 - iii. Updation

```

// </> syntax
for ( initialization; condition; updation){
    //code
}
// flow

```

```
// Inititalization -> condition -> code -> updation

// example
for (let i =1 ; i<=5; i++){
    document.writeIn("<h1>Hey</h1>");
}
```

▼ while loop

- The while loop is used when the number of iterations is not known
- The condition is checked first.
- If the condition is `true` , the code executes.

```
// </> syntax
while(condition){
    // code
    // updation
}

// flow
initialization --> condition -> code -> updation

//example
let i = 0
while(i<=5){
    console.log("hello");
    i++;
}
```

▼ do while loop

- The do while loop executes the code at least one time even if the condition is false.
- Condition is checked after execution.

```
initialization
do{
    code
    updation
}
```

```

}
while(condition)

/*Flow:
// Initialization --> Code --> Updation --> Condi
on

/*Example:
let k = 0;
do{
    document.writeln("<h3>Lets Breakup💔</h3>")
    k++
}while(k>=2)

```

▼ for in loop

- The for in loop is used to iterate over object keys.

```

// </> syntax
for ( variable in object){
    // code
}

// example
let detials = {
    name: "pavan",
    age: 26,
    isEmployee: true,
}

for ( let keys in details){
    console.log(keys) // name, age, isEmployee
    console.log(details[keys]) // 'pavan', //26, //tru
e
}

```

▼ for of loop

- The for of loop is used to iterate over iterable values like:
 - arrays
 - Strings

```

// </> syntax
for ( variable of iterable){
    // code
}

// example
// string
let str = "monthy"
for ( let values of str){
    console.log(values)
}

// array
let arr = ["html", "css", "js"]
for ( let values of arr){
    console.log(values) // html //css //js
}

```

▼ Core Concepts

- ▼ Functions
- ▼ Hoisting and TDZ(Temporal Dead Zone)
- ▼ Difference b/w var, let, and const
- ▼ Execution Context
- ▼ Closures
- ▼ Strings
- ▼ Arrays
- ▼ Objects
- ▼ Math and Date object
- ▼ Destructuring
- ▼ Spread operator and rest parameter

▼ Advanced and Asynchronous JavaScript

- ▼ `setTimeout()` and `setInterval()`
- ▼ Bom (Browser object model)
- ▼ Dom (Document object model)
- ▼ Events and tis propagation
- ▼ Promises
- ▼ Async and await
- ▼ Fetch API
- ▼ Form Validation